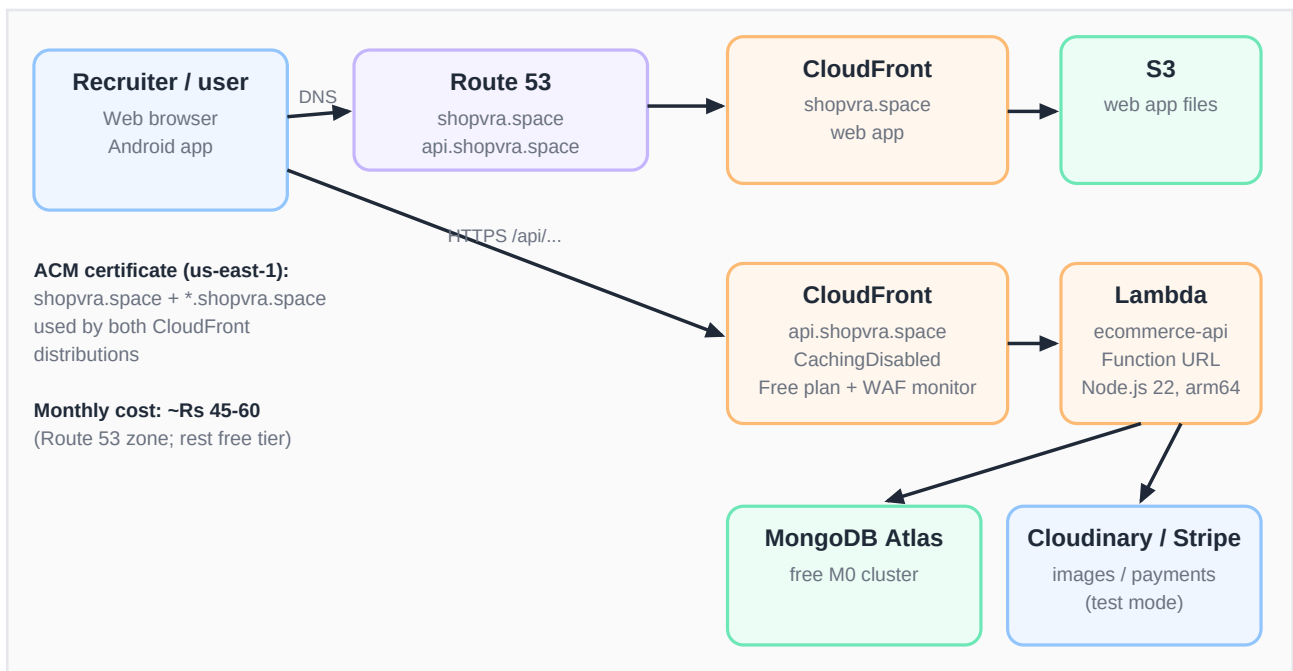


Deploy the Express.js E-commerce Backend on AWS

Complete step-by-step guide with real screenshots - AWS Lambda + CloudFront + Route 53 + ACM, live at <https://api.shopvra.space>

Result: the backend in `backend/expressJs/expressJs_ecommerce_backend` runs on AWS Lambda behind CloudFront on your own domain with HTTPS, for about **Rs 0 per month** for the backend itself. The web app (`shopvra.space`) and the mobile app both call it. Verified live: **18/18** shopper-flow requests passed.

Architecture



Contents

1. Your deployment at a glance (all values)
2. Prerequisites
3. Prepare the backend code for Lambda
4. Prepare MongoDB Atlas
5. Create the Lambda function
6. Configure Lambda (memory, timeout, environment variables)
7. Create the Function URL and test it
8. Create the CloudFront distribution for `api.shopvra.space`
9. Route 53 DNS record
10. Test the live API (and fix 'site can't be reached')
11. Point the web app and mobile app at the API
12. Stripe webhook
13. Budget alert and logs
14. Deploying updates later
15. Cost
16. Troubleshooting (including real issues we hit)
17. Final checklist

1. Your deployment at a glance

Every value used in this deployment (none of them are secrets). Keep this page for reference.

Item	Value
Live API URL	https://api.shopvra.space
AWS region	US East (N. Virginia) - us-east-1
Lambda function	ecommerce-api - Node.js 22.x, arm64, 512 MB, 30 s timeout, handler index.handler
Lambda Function URL	https://lq6ezfzwi7ilwmpmaoxnnjexoi0okbnb.lambda-url.us-east-1.on.aws/ (auth NONE)
Upload file	lambda.zip (~795 KB), built with <code>npm run build:lambda</code>
CloudFront distribution	ecommerce-backend - ID EIY8R7XTOCFR6 - Free plan (\$0/month)
CloudFront domain	d2czc93szwjsn9.cloudfront.net
CloudFront behaviour	Redirect HTTP to HTTPS; all methods; CachingDisabled; AllViewerExceptHostHeader; WAF monitor mode
TLS certificate (ACM, us-east-1)	arn:aws:acm:us-east-1:084767318696:certificate/550d0388-ba3b-4dce-87f9-fefb9e86b181 - covers shopvra.space and *.shopvra.space
Route 53	Hosted zone shopvra.space - records api (A + AAAA, alias to CloudFront)
Domain nameservers (set at GoDaddy)	ns-110.awsdns-13.com, ns-1377.awsdns-44.org, ns-1579.awsdns-05.co.uk, ns-647.awsdns-16.net
Database	MongoDB Atlas free M0 cluster (Network Access 0.0.0.0/0)
Web app	https://shopvra.space (S3 + CloudFront) - production build calls https://api.shopvra.space
Mobile app	Release builds (eas build) call https://api.shopvra.space
Allowed websites (CORS)	https://shopvra.space , https://www.shopvra.space (mobile app always allowed)

Secrets are never written in this guide. MongoDB URI, JWT secret, Cloudinary keys, Stripe keys and the admin password live only in your local `.env` and in the Lambda environment variables. In the screenshots they are blacked out.

2. Prerequisites

- **AWS account** (this guide uses account 'ritikeducated', region us-east-1).
- **Domain** shopvra.space bought at GoDaddy, with its nameservers changed to the four Route 53 nameservers above, so Route 53 manages its DNS (already done when the web app was deployed).
- **ACM certificate** in **us-east-1** for shopvra.space and *.shopvra.space, status **Issued** (CloudFront only accepts certificates from us-east-1). The wildcard also covers api.shopvra.space.
- **MongoDB Atlas** cluster (free M0), **Cloudinary** account and **Stripe** account in test mode.
- **Node.js 22** on your computer, and optionally the **AWS CLI** for uploading builds.

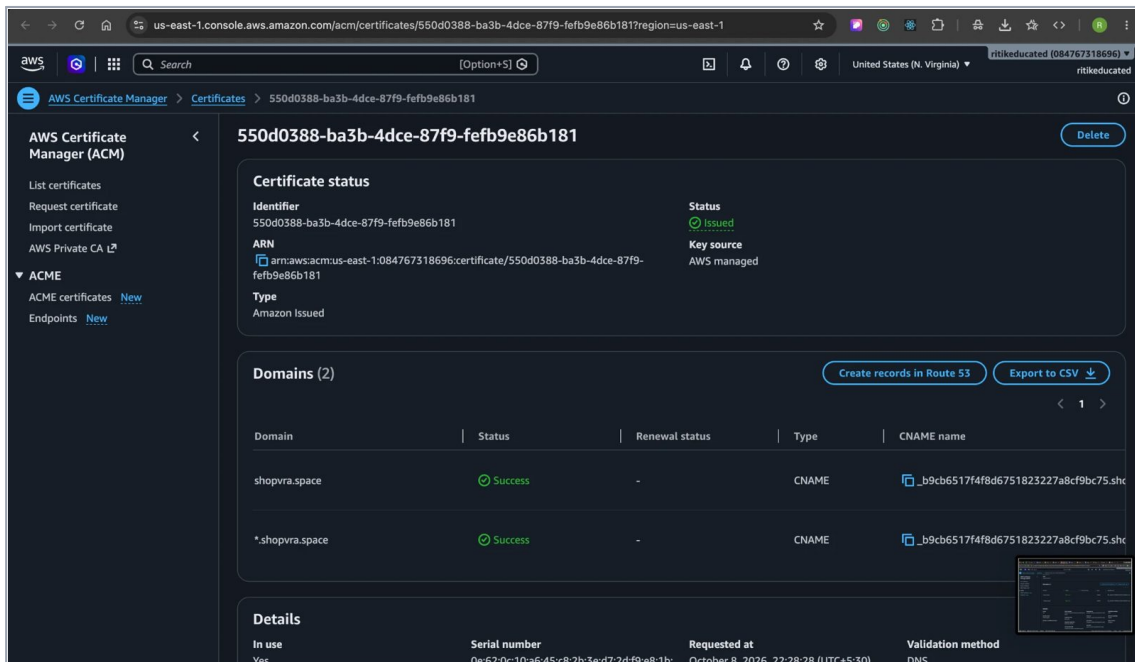


Figure 1. The existing ACM certificate (us-east-1): status Issued, covering shopvra.space and *.shopvra.space, validated by DNS.

3. Prepare the backend code for Lambda

These code changes are already made. Running locally works exactly as before (`npm start`).

File	What it does
<code>app.ts (new)</code>	Builds the Express app (routes, CORS, Stripe webhook, JSON). Shared by the local server and Lambda. JSON 404 and error responses (no stack traces).
<code>server.ts</code>	Local entry point: loads <code>.env</code> , connects, <code>app.listen</code> on PORT 3000.
<code>lambda.ts (new)</code>	Lambda entry point <code>index.handler</code> using <code>serverless-http</code> , with a fix so Express 5 reads JSON, multipart and Stripe raw bodies.
<code>config/startup.ts (new)</code>	Connects to MongoDB, syncs indexes and creates the admin once per Lambda container; later requests reuse the connection.
<code>config/db.ts</code>	Throws instead of <code>process.exit()</code> so a failed connection doesn't kill the container.
<code>scripts/build-lambda.mjs (new)</code>	esbuild bundles everything into one file and creates <code>lambda.zip</code> .
<code>package.json</code>	Scripts <code>build:lambda</code> (<code>lambda.zip</code>) and <code>start:prod</code> (<code>node dist/server.js</code>).
<code>.env.example</code>	CORS_ORIGINS - only listed websites may call the API.

Build the upload file

```
cd backend/expressJs/expressJs_ecommerce_backend
npm install
npm run build:lambda
```

lambda.zip ready (794 KB). Upload it to Lambda; handler: `index.handler`, runtime: `Node.js 22.x`

Tested before upload

The bundled code was invoked with the exact request format Lambda Function URLs send:

```

PASS 200 2382 ms GET / (cold start: connects to MongoDB)
PASS 200 1 ms GET / (warm)
PASS 200 209 ms GET /api/products
PASS 204 1 ms CORS preflight from https://shopvra.space
PASS 204 0 ms CORS blocks other websites
PASS 200 200 ms POST /api/auth/login (admin)
PASS 200 131 ms GET /api/auth/me (Bearer token)
PASS 200 1059 ms GET /api/admin/stats
PASS 200 3759 ms GET /api/orders/admin/all
PASS 201 10395 ms POST /api/products multipart image (53 KB, base64)
PASS 200 1478 ms DELETE /api/products/:id (cleanup)
PASS 200 108 ms POST /api/stripe webhook (signed, raw body)
PASS 400 3 ms POST /api/stripe webhook (bad signature rejected)
PASS 404 1 ms Unknown route returns JSON 404
PASS 400 2 ms Invalid JSON body returns JSON 400 (no stack trace)

```

==== 15/15 passed ====

The local server also passed the full Postman collection: 43/43 requests.

4. Prepare MongoDB Atlas

Lambda has no fixed IP address, so Atlas must accept connections from any IP. Protect it with a strong database password.

MongoDB Atlas › Security › Network Access › Add IP Address

Access List Entry	0.0.0.0/0 (Allow access from anywhere)
Comment	AWS Lambda

MongoDB Atlas › Security › Database Access › Add New Database User (recommended)

Authentication	Password
Username / password	a dedicated user with a long generated password
Role	Read and write to any database
Use this user in MONGODB_URI. Recommended: a separate database name for production (e.g. /ecommerce-prod).	

5. Create the Lambda function

Open **Lambda** in region **United States (N. Virginia)** and click **Create function**.

AWS Console › Lambda › Functions › Create function

Option	Author from scratch
Function name	ecommerce-api
Runtime	Node.js 22.x
Additional settings › ARM64 architecture	ON (cheaper)
Durable execution / EC2 capacity provider	OFF
Permissions	Default - Lambda creates a role that can write CloudWatch logs

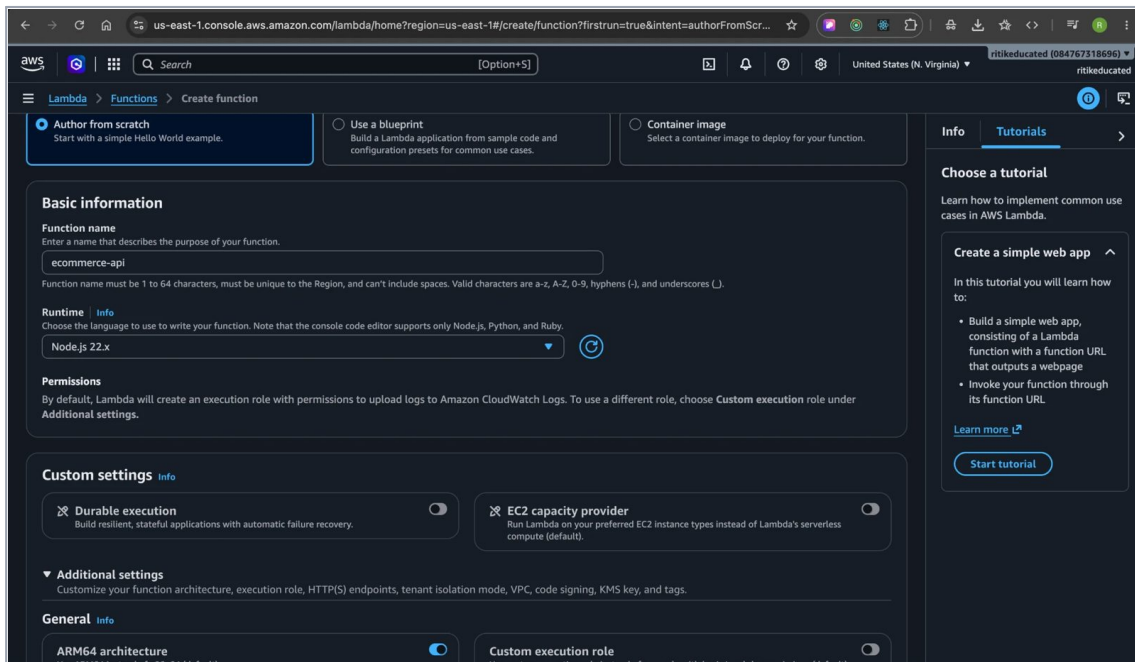


Figure 2. Create function: Author from scratch, name ecommerce-api, runtime Node.js 22.x, ARM64 architecture ON.

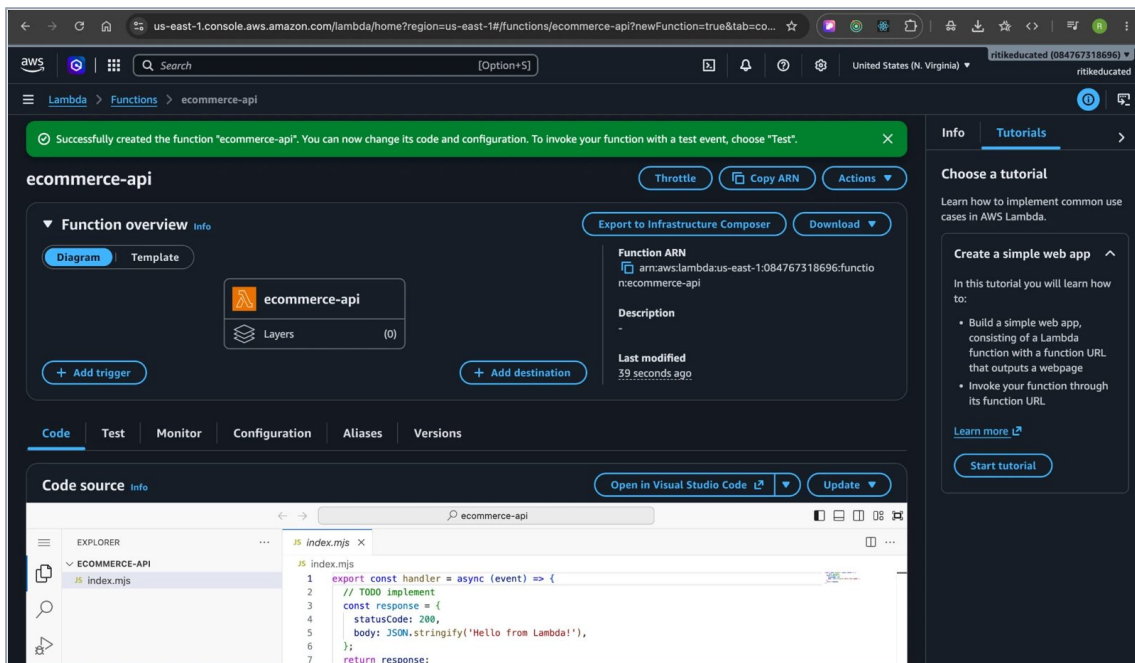


Figure 3. Function created. The default code is a 'Hello from Lambda' example - it is replaced in the next step.

Upload the code

- 1 On the **Code** tab click **Upload from > .zip file**.
- 2 Choose backend/expressJs/expressJs_ecommerce_backend/lambda.zip and click **Save**.
- 3 Keep the handler as `index.handler` (Code > Runtime settings).

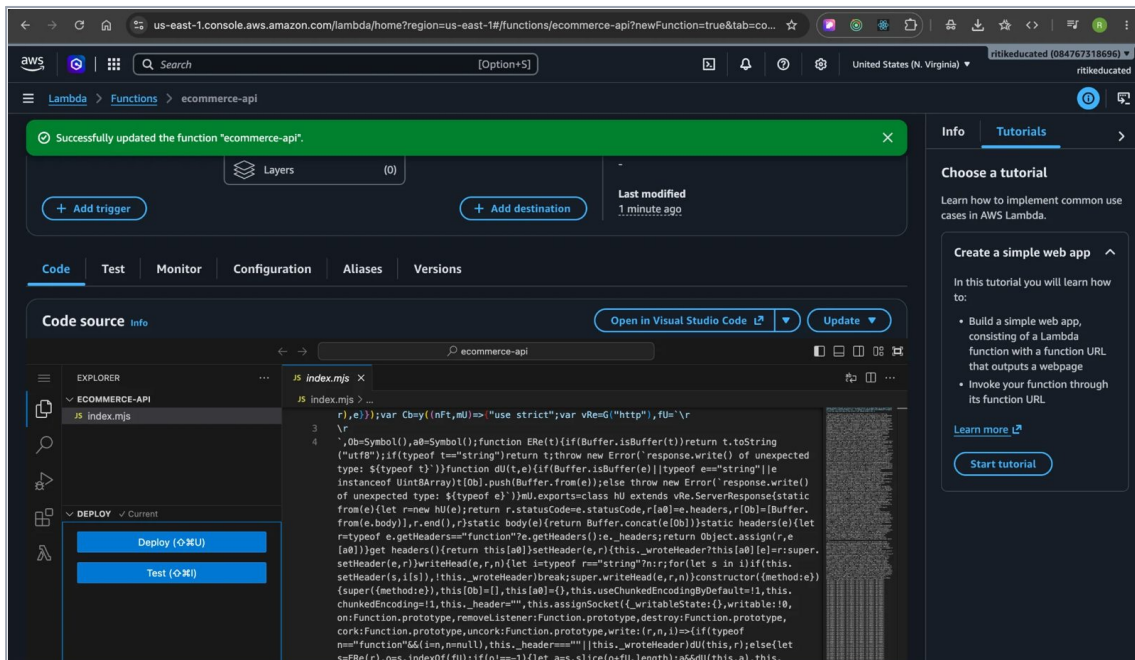


Figure 4. After uploading lambda.zip: 'Successfully updated the function'. The editor shows the bundled index.mjs and DEPLOY shows 'Current'.

Do not edit the code in the browser editor - it is the minified bundle. Change code on your computer, run `npm run build:lambda` and upload the new zip. You don't need to click Deploy or Test after uploading a zip.

6. Configure Lambda

General configuration

AWS Console › Lambda › ecommerce-api › Configuration › General configuration › Edit	
Memory	512 MB
Ephemeral storage	512 MB (default)
Timeout	0 min 30 sec
SnapStart	None

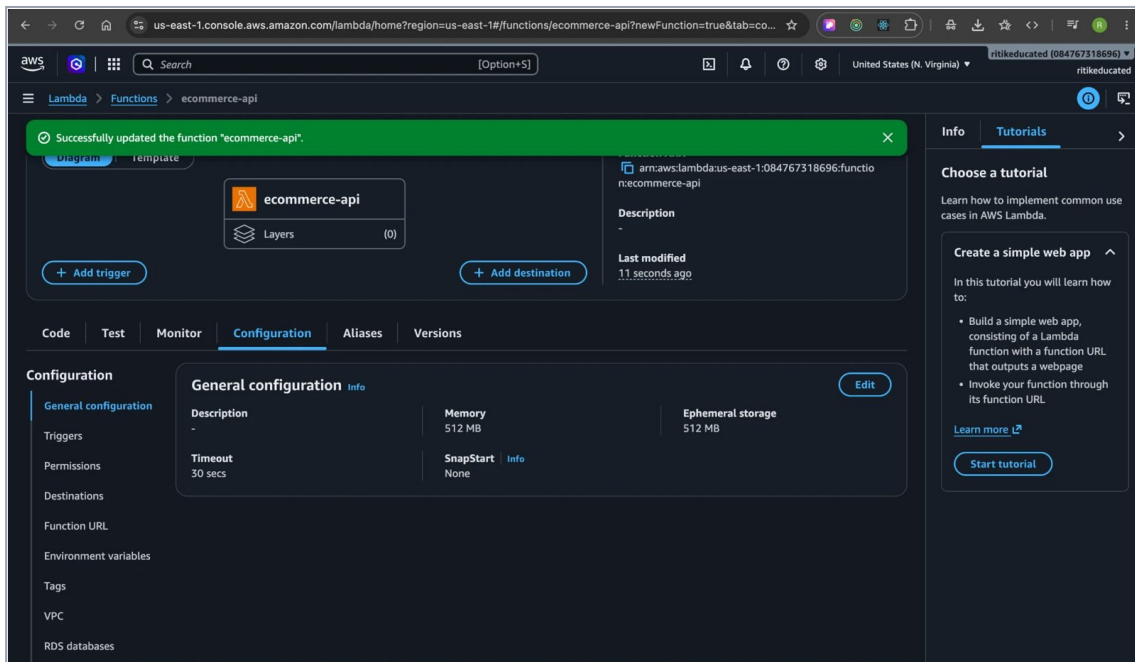


Figure 5. General configuration saved: Memory 512 MB, Ephemeral storage 512 MB, Timeout 30 secs.

Environment variables

Configuration > Environment variables > Edit > **Add environment variable** for each key. Paste values without quotes. Do **not** add PORT.

Key	Value
MONGODB_URI	your Atlas connection string
JWT_SECRET	a long random string - generate with <code>openssl rand -hex 32</code> (never the placeholder)
JWT_EXPIRE	7d
CLOUDINARY_CLOUD_NAME / CLOUDINARY_API_KEY / CLOUDINARY_API_SECRET	from your Cloudinary dashboard
STRIPE_SECRET_KEY	sk_test_... (test mode)
STRIPE_WEBHOOK_SECRET	whsec_... from step 12 (placeholder until then)
ADMIN_EMAIL / ADMIN_PASSWORD	admin account for the mobile admin panel
CORS_ORIGINS	https://shopvra.space, https://www.shopvra.space
NODE_ENV	production

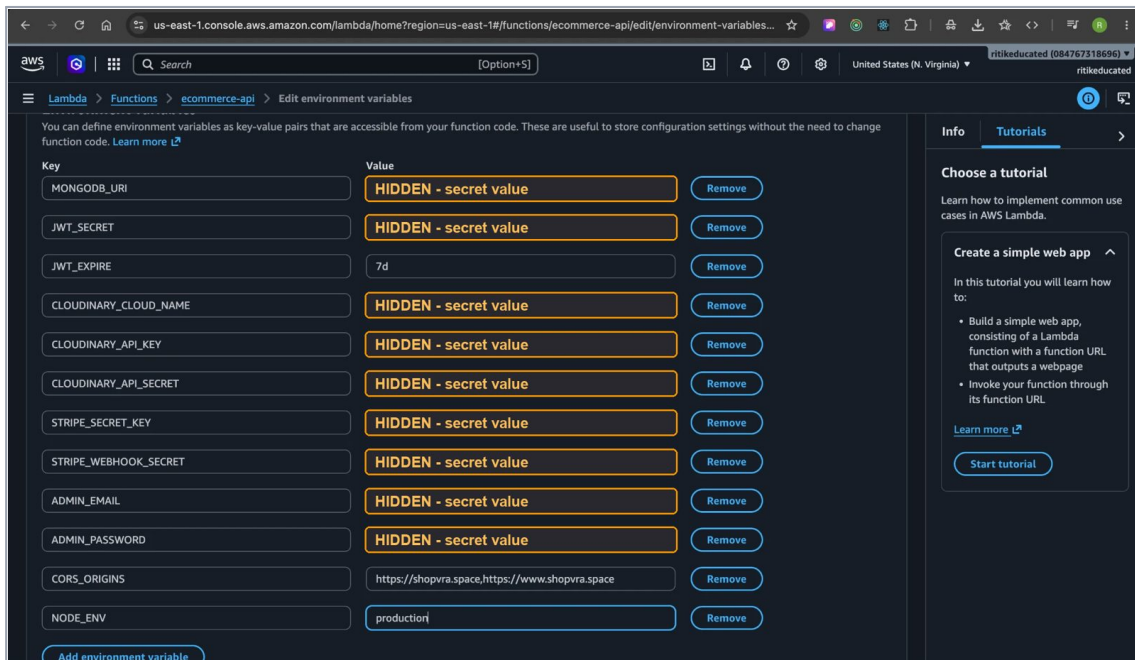


Figure 6. Environment variables (secret values blacked out). Only JWT_EXPIRE, CORS_ORIGINS and NODE_ENV are shown.

Important: in this screenshot JWT_SECRET still had the placeholder value. Replace it with the output of `openssl rand -hex 32`, otherwise anyone could create admin login tokens. Changing ADMIN_PASSWORD here does not change the password of an admin that already exists in the database.

7. Create the Function URL and test it

AWS Console > Lambda > ecommerce-api > Configuration > Function URL > Create function URL

Auth type	NONE
Invoke mode (Additional settings)	BUFFERED (default)
Configure cross-origin resource sharing (CORS)	Unchecked - Express already handles CORS

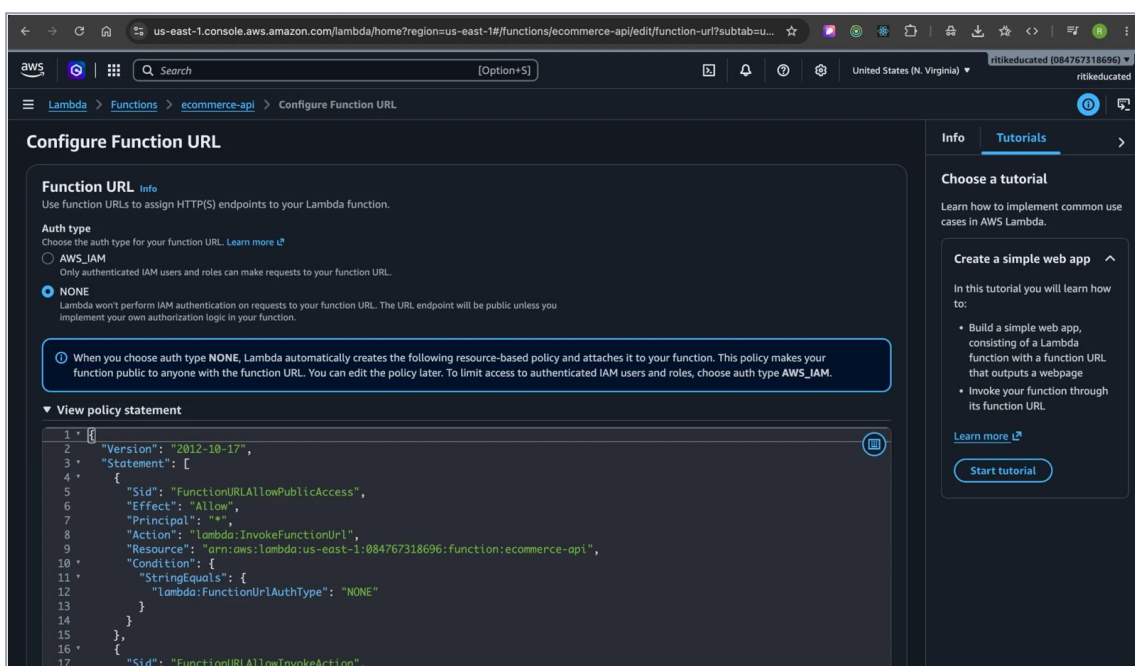


Figure 7. Auth type NONE. Lambda automatically adds a resource-based policy allowing public access to the Function URL.

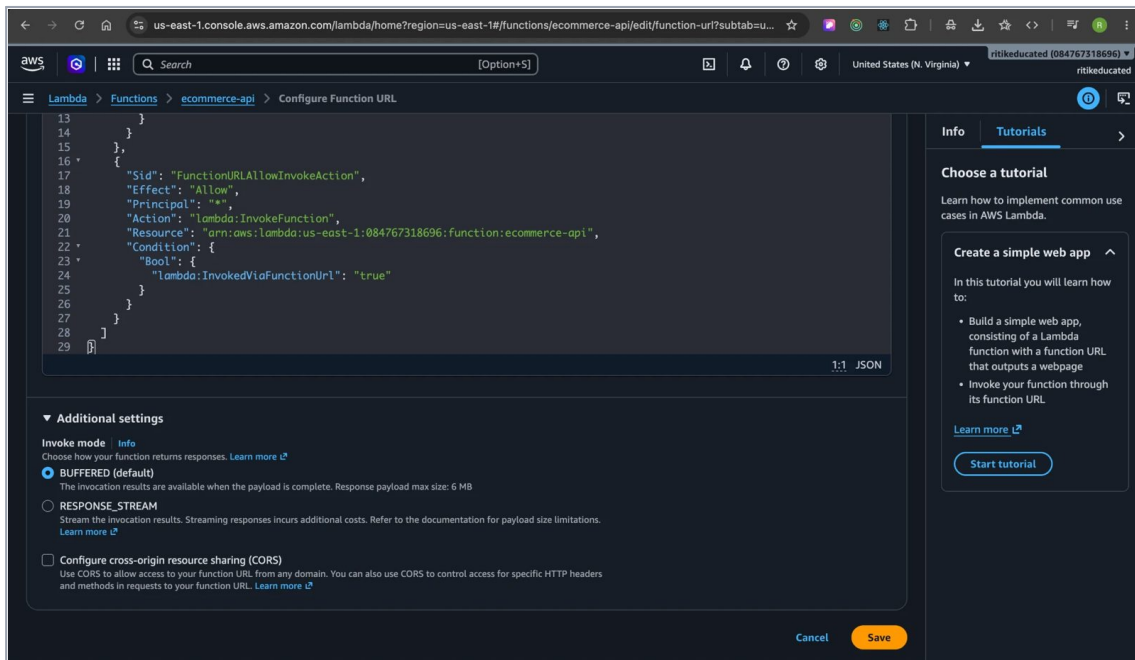


Figure 8. Additional settings: Invoke mode BUFFERED, CORS left unchecked. Click Save.

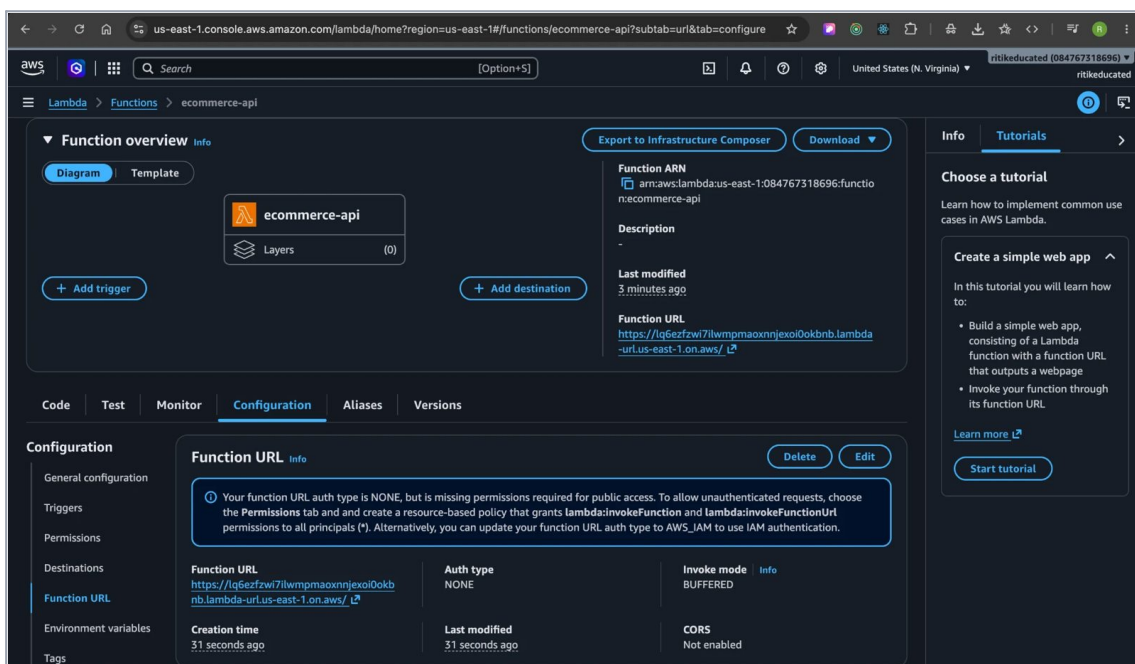


Figure 9. Function URL created. The blue notice about missing permissions can be ignored - the URL works (next figure).

Test: open the Function URL in a browser. The first request takes 1-3 seconds while it connects to MongoDB.

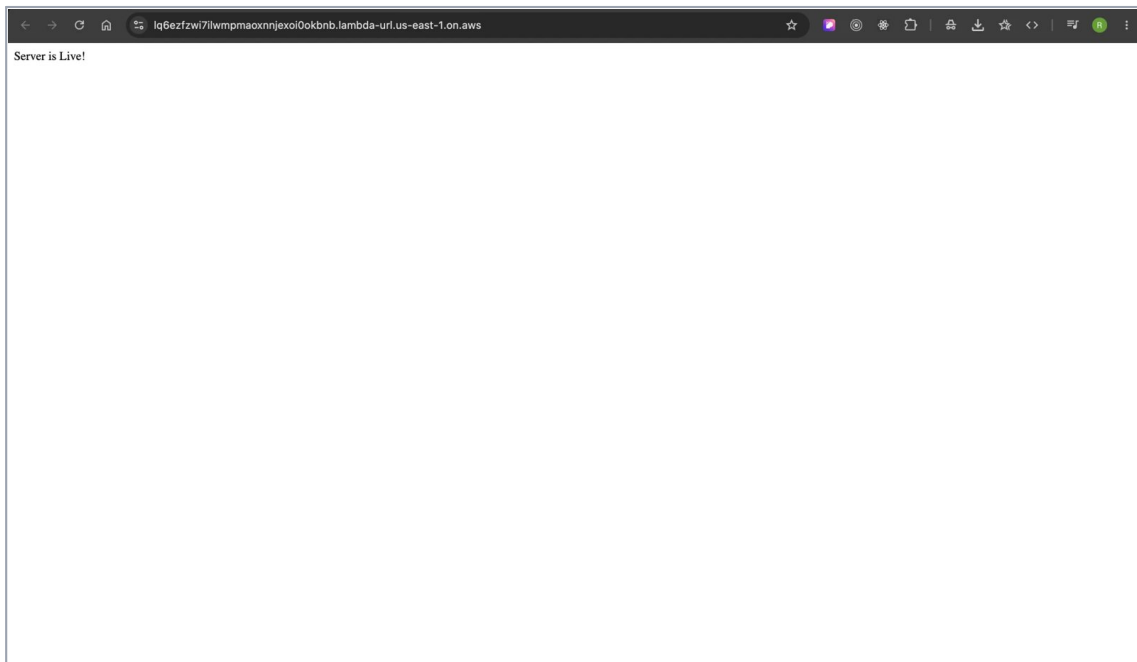


Figure 10. The Function URL returns 'Server is Live!' - Lambda is running the Express app.

```
curl https://lq6ezfzwi7ilwmpmaoxnnjexoi0okbnb.lambda-url.us-east-1.on.aws/  
# Server is Live!  
curl "https://lq6ezfzwi7ilwmpmaoxnnjexoi0okbnb.lambda-url.us-east-1.on.aws/api/products?limit=3"  
# {"success":true,"data":[...]}
```

8. Create the CloudFront distribution for api.shopvra.space

CloudFront gives the API your own domain and certificate. Open **CloudFront** > **Distributions** > **Create distribution**. The console is a 6-step wizard; Step 1 (Choose a plan) uses the **Free plan (\$0/month)**.

Step 2 - Get started

AWS Console > CloudFront > Create distribution > Get started	
Distribution name	ecommerce-backend
Description	optional
Distribution type	Single website configuration
Route 53 managed domain	api.shopvra.space > click Check domain

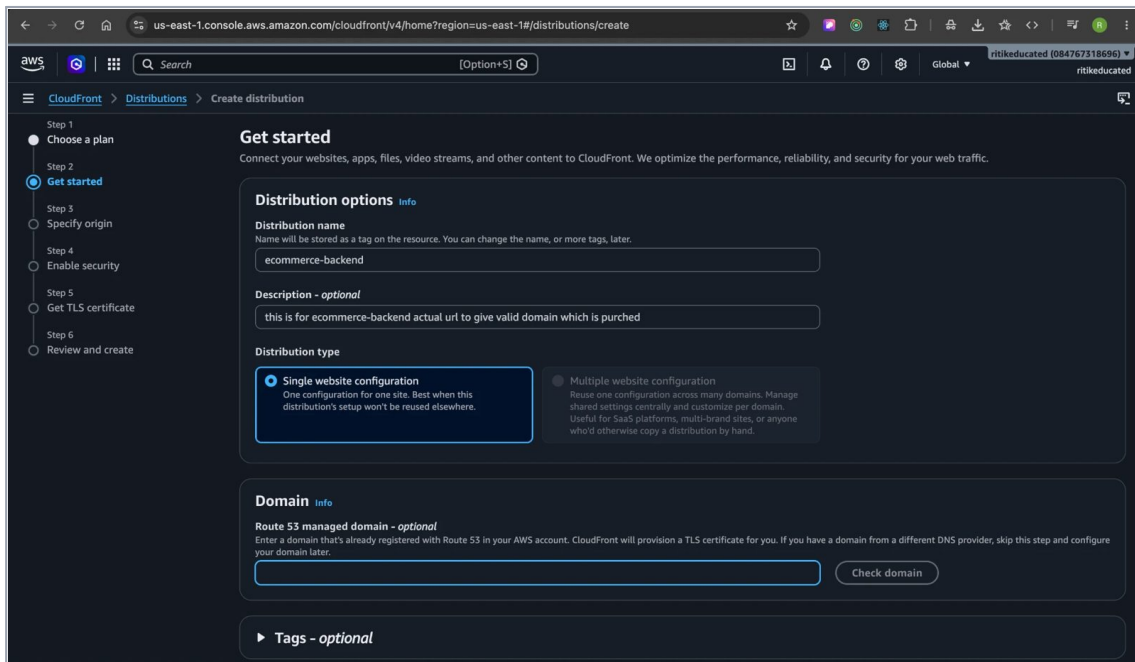


Figure 11. Get started: distribution name ecommerce-backend, Single website configuration.

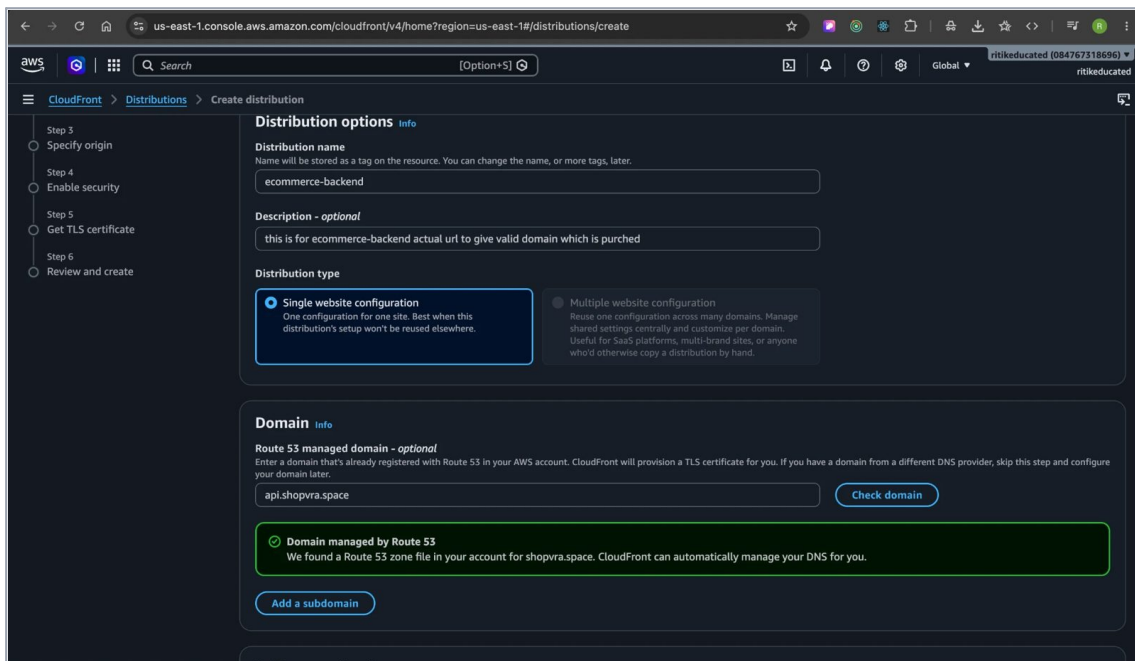


Figure 12. Check domain: 'Domain managed by Route 53' - CloudFront can use your certificate and manage the DNS record.

Step 3 - Specify origin

AWS Console › CloudFront › Create distribution › Specify origin

Origin type	Other
Custom origin	lq6ezfzwi7ilwmpmaoxnnjexoi0okbnb.lambda-url.us-east-1.on.aws (no https:// and no /)
Origin path	(empty)
Allow private Lambda function access to CloudFront	Unchecked (see warning below)

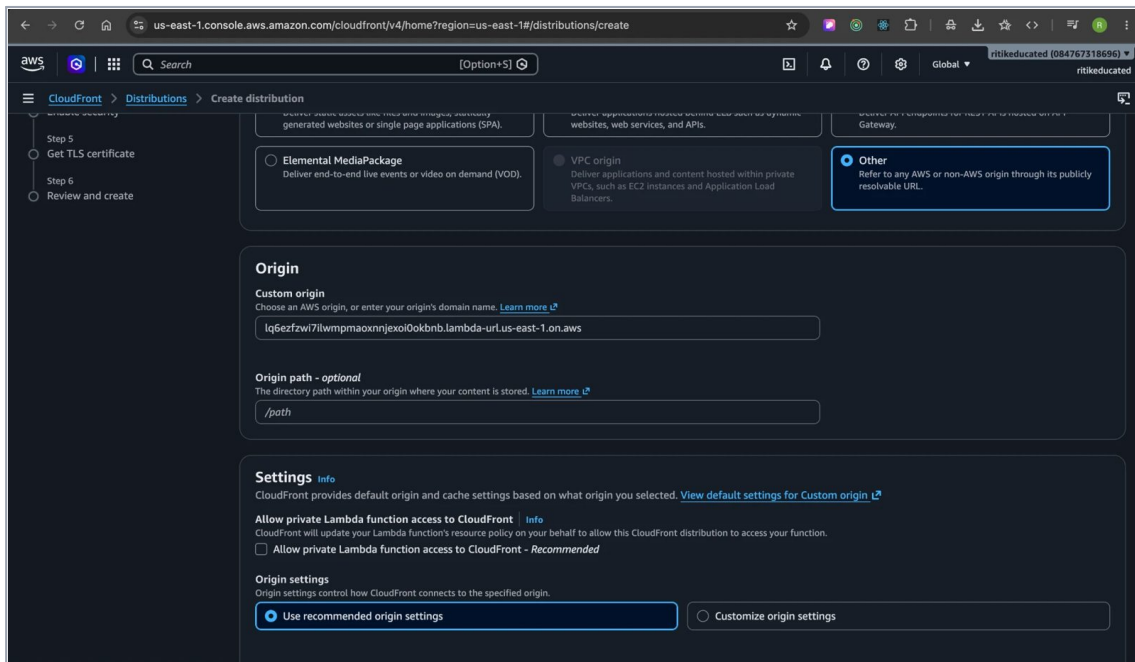


Figure 13. Origin type Other with the Function URL host as the custom origin.

Leave 'Allow private Lambda function access to CloudFront' unchecked, even though it says Recommended. It signs requests to Lambda, and then POST/PUT requests only work if the app sends an x-amz-content-sha256 header - login, cart, orders and uploads would fail.

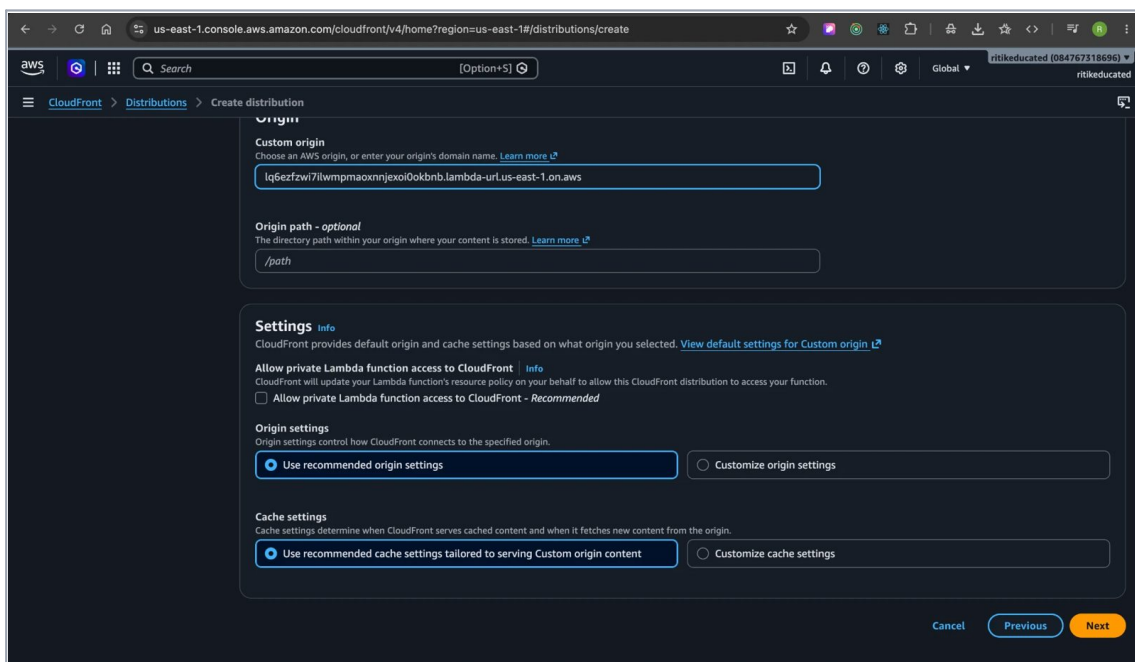


Figure 14. Settings section: private Lambda access unchecked. Origin settings and Cache settings are changed to Customize next.

AWS Console › Specify origin › Origin settings › Customize origin settings

Origin mutual TLS / Origin Shield	OFF (paid plans)
Protocol	HTTPS only
HTTPS port	443
Minimum Origin SSL protocol	TLSv1.2
Connection attempts / timeout	3 / 10 (defaults)

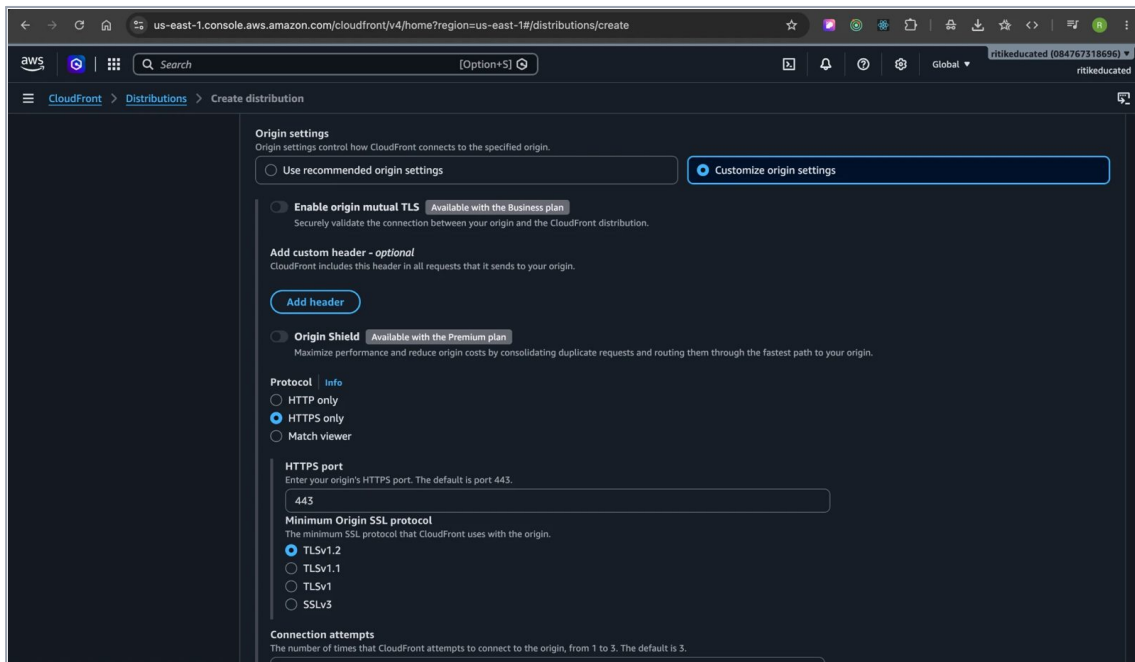


Figure 15. Customize origin settings: Protocol HTTPS only, port 443, minimum TLSv1.2.

AWS Console › Specify origin › Cache settings › Customize cache settings	
Viewer protocol policy	Redirect HTTP to HTTPS
Allowed HTTP methods	GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE
Cache HTTP methods - OPTIONS	Unchecked
Allow gRPC requests over HTTP/2	Unchecked
Cache policy	CachingDisabled (labelled 'Recommended for Lambda Function URLs')
Origin request policy	AllViewerExceptHostHeader (required - the Function URL rejects other Host headers)
Response headers policy	(none)

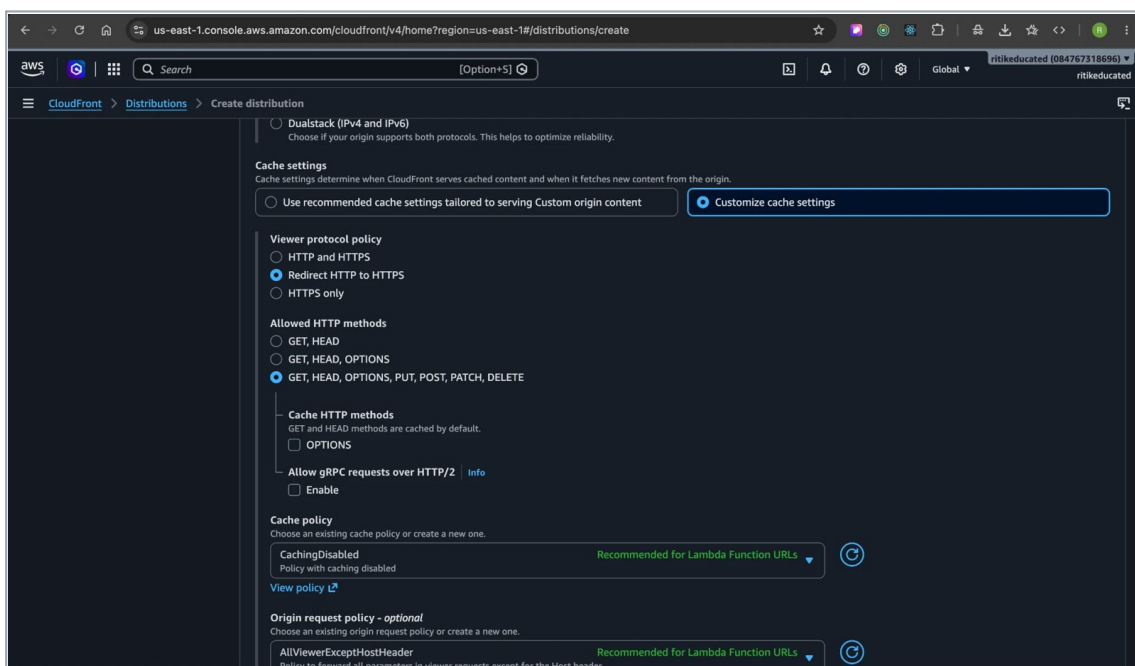


Figure 16. Customize cache settings: all HTTP methods, CachingDisabled and AllViewerExceptHostHeader.

Step 4 - Enable security

AWS Console › CloudFront › Create distribution › Enable security

Web Application Firewall (WAF)	Included in the Free plan (cannot be turned off)
Use monitor mode	Checked - counts but never blocks requests
Protection against Layer 7 DDoS attacks	OFF (Business plan)

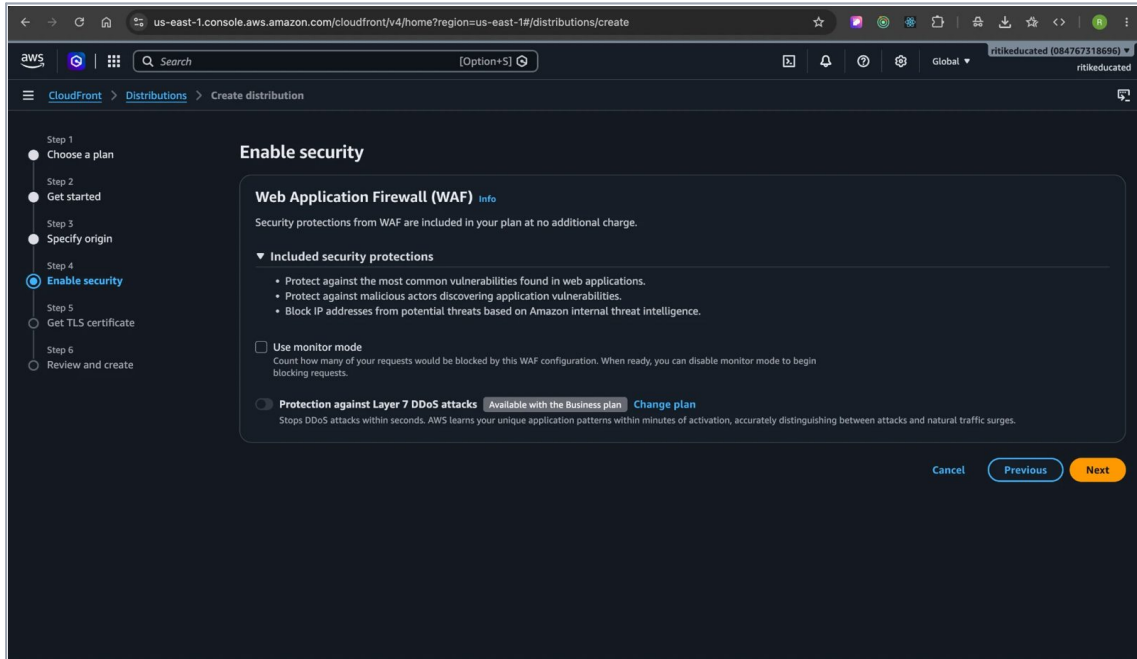


Figure 17. Enable security. Tick 'Use monitor mode': the firewall's body-size rule (about 8 KB) would otherwise block product image uploads.

Step 5 - Get TLS certificate

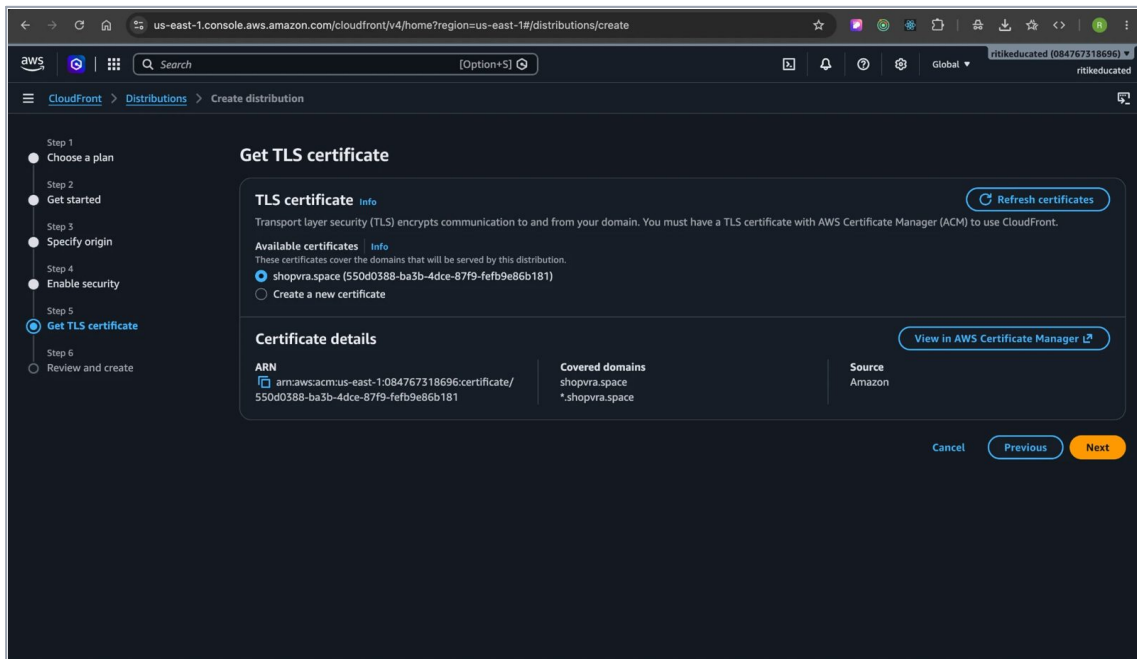


Figure 18. Choose the existing certificate shopvra.space (covers shopvra.space and *.shopvra.space). No new certificate is needed.

Step 6 - Review and create

Check: Billing Free (\$0/month), Domains to serve `api.shopvra.space`, Custom origin = Function URL, Grant CloudFront access to origin = No, Cache policy `CachingDisabled`, Origin request policy `AllViewerExceptHostHeader`, Use monitor mode Yes.

The screenshot shows the 'Review and create' step in the AWS CloudFront console. The left sidebar lists steps from 'Choose a plan' to 'Review and create', with 'Review and create' being the active step. The main content area is divided into three sections: 'General configuration', 'Origin', and 'Cache settings'. Each section contains several configuration items with 'Edit' buttons. The 'General configuration' section shows 'Distribution name' as 'ecommerce-backend', 'Description' as 'this is for ecommerce-backend actual url to give valid domain which is purchsed', 'Domains to serve' as 'api.shopvra.space', and 'Billing' as 'Free (\$0/month)'. The 'Origin' section shows 'Custom origin' as 'lq6ezfzw7ilwmpmaoxnjexoi0okbnb.lambd-us-east-1.on.aws', 'Origin path' as '-', 'Grant CloudFront access to origin' as 'No', 'Enable Origin Shield' as 'No', 'Connection attempts' as '3', and 'Connection timeout' as '10'. The 'Cache settings' section shows 'Viewer protocol policy' as 'Redirect HTTP to HTTPS', 'Allowed HTTP methods' as 'GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE', 'Cache policy' as 'CachingDisabled', 'Origin request policy' as 'AllViewerExceptHostHeader', and 'Response headers policy' as '-'. At the bottom right, there are 'Cancel', 'Previous', and 'Create distribution' buttons.

Figure 19. Review: Free plan, `api.shopvra.space`, Function URL origin, CloudFront access to origin No, all methods, `CachingDisabled`, `AllViewerExceptHostHeader`.

This screenshot shows the bottom portion of the 'Review and create' step. It includes the 'Connection attempts' and 'Connection timeout' settings from the previous section. The 'Cache settings' section is repeated. Below it is the 'Security' section, which shows 'Security protections' as 'Enabled' and 'Use monitor mode' as 'Yes'. The 'TLS certificate' section shows 'ARN' as 'arn:aws:acm:us-east-1:084767318696:certificate/550d0388-ba3b-4dce-87f9-feb9e86b181', 'Covered domains' as 'shopvra.space' and '*.shopvra.space', and 'Source' as 'Amazon'. At the bottom right, there are 'Cancel', 'Previous', and 'Create distribution' buttons.

Figure 20. Review (bottom): Security Enabled with Use monitor mode Yes, certificate covering `shopvra.space` and `*.shopvra.space`. Click Create distribution.

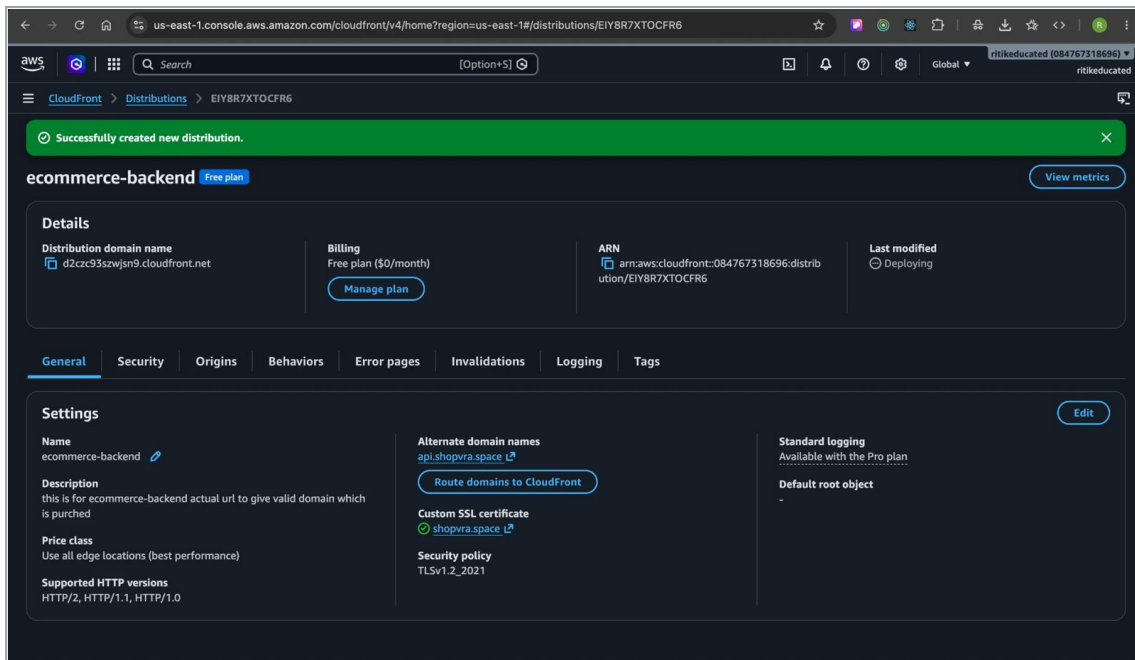


Figure 21. Distribution created: ecommerce-backend (Free plan), domain d2czc93szwjsn9.cloudfront.net, alternate domain api.shopvra.space. Last modified shows 'Deploying' for 5-10 minutes.

9. Route 53 DNS record

The distribution does not answer on api.shopvra.space until a DNS record points to it. On the distribution page click **Route domains to CloudFront**, then **Set up routing automatically**.

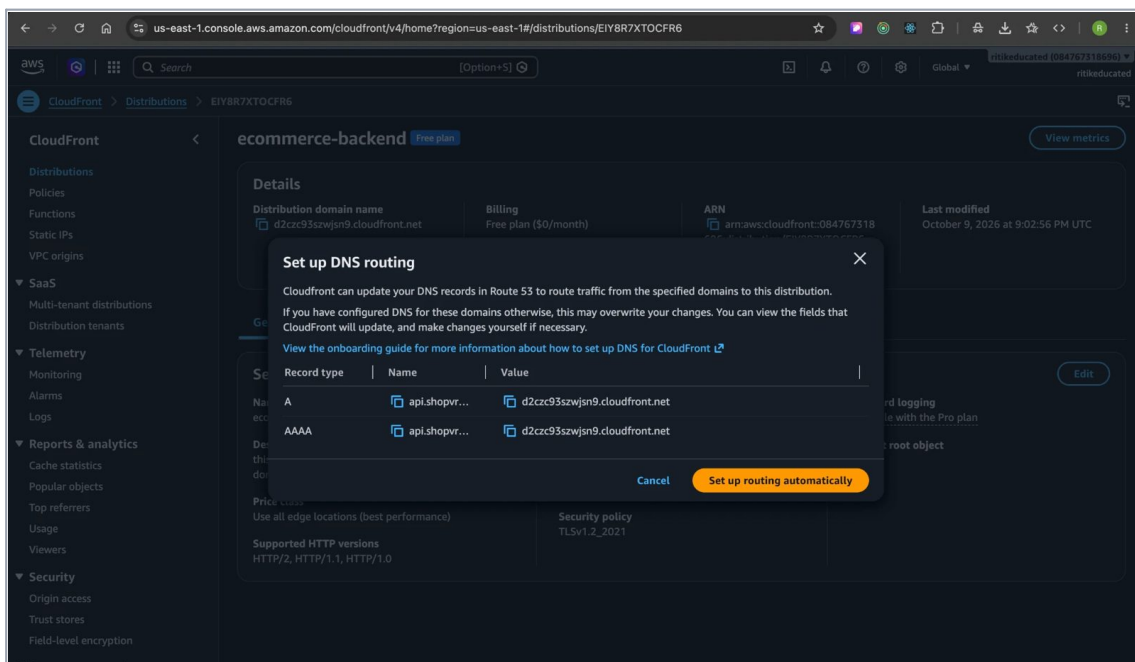


Figure 22. Set up DNS routing: CloudFront creates A and AAAA alias records for api.shopvra.space pointing to d2czc93szwjsn9.cloudfront.net.

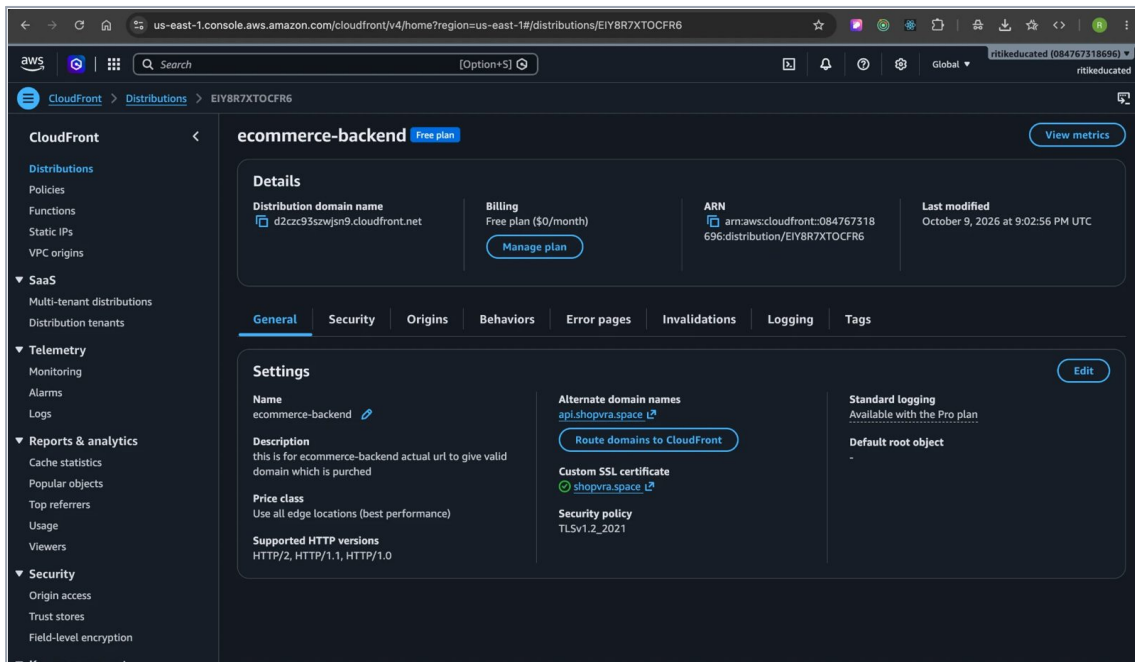


Figure 23. After routing is set up and deployment finished: Last modified shows a date (deployed).

Do not click Cancel in that dialog - that is what happened first in this deployment, so api.shopvra.space did not resolve until routing was set up. Existing shopvra.space records are not changed.

AWS Console › Route 53 › Hosted zones › shopvra.space › Create record (manual alternative)

Record name	api
Record type	A
Alias	ON
Route traffic to	Alias to CloudFront distribution › d2czc93szwjsn9.cloudfront.net
Routing policy	Simple routing

Optionally repeat with type AAAA for IPv6.

Check the record from your terminal:

```
dig +short @8.8.8.8 api.shopvra.space A
# 18.172.78.x (CloudFront addresses = record is live)
```

10. Test the live API

```
curl https://api.shopvra.space/
# Server is Live!

curl "https://api.shopvra.space/api/products?limit=2"
# {"success":true,"data":[...]}

curl https://api.shopvra.space/api/nope
# {"success":false,"message":"Route not found: GET /api/nope"}

curl http://api.shopvra.space/      # 301 -> https://api.shopvra.space/
```

Live check (api.shopvra.space)	Result
GET /	200 Server is Live!
GET /api/products	200 - 12 products from MongoDB
POST /api/auth/login (wrong password)	JSON body reaches Express - 'Invalid credentials'

Live check (api.shopvra.space)	Result
Protected route without token	401 Not authorized, no token
CORS from https://shopvra.space / other site	allowed / blocked
HTTP	301 redirect to HTTPS
Certificate	CN=shopvra.space, valid until Apr 2027
Full shopper flow (web + mobile style)	18/18 passed: register, login, products, cart, wishlist, address, cash order, my orders

If your browser says 'This site can't be reached'

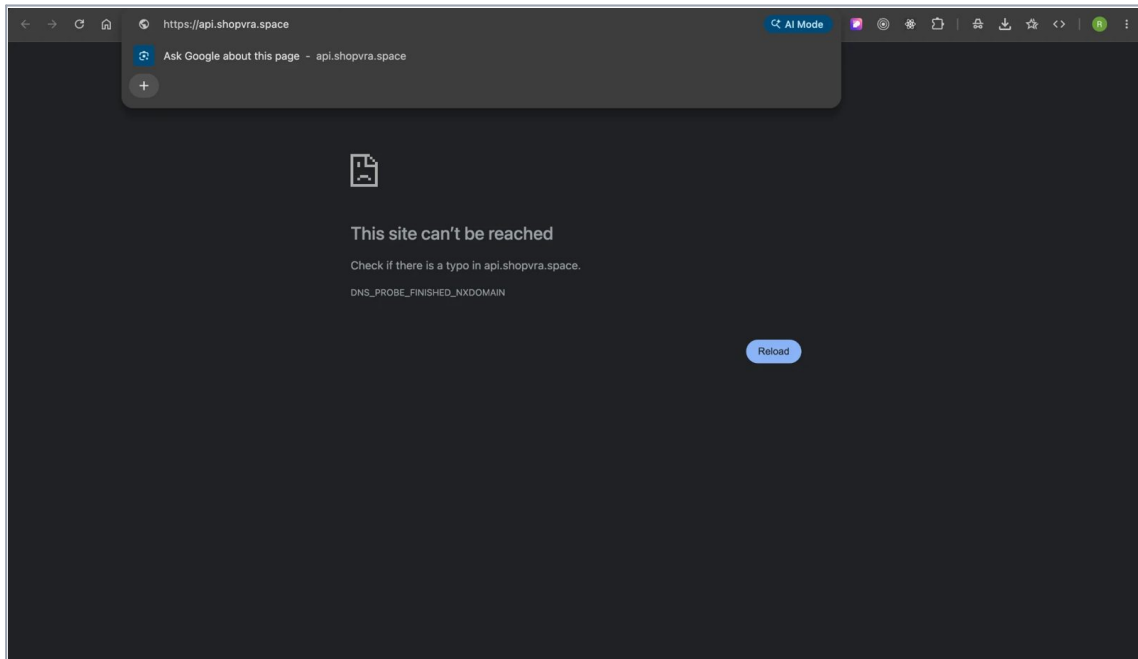


Figure 24. DNS_PROBE_FINISHED_NXDOMAIN: your network looked up api.shopvra.space before the record existed and cached 'does not exist'.

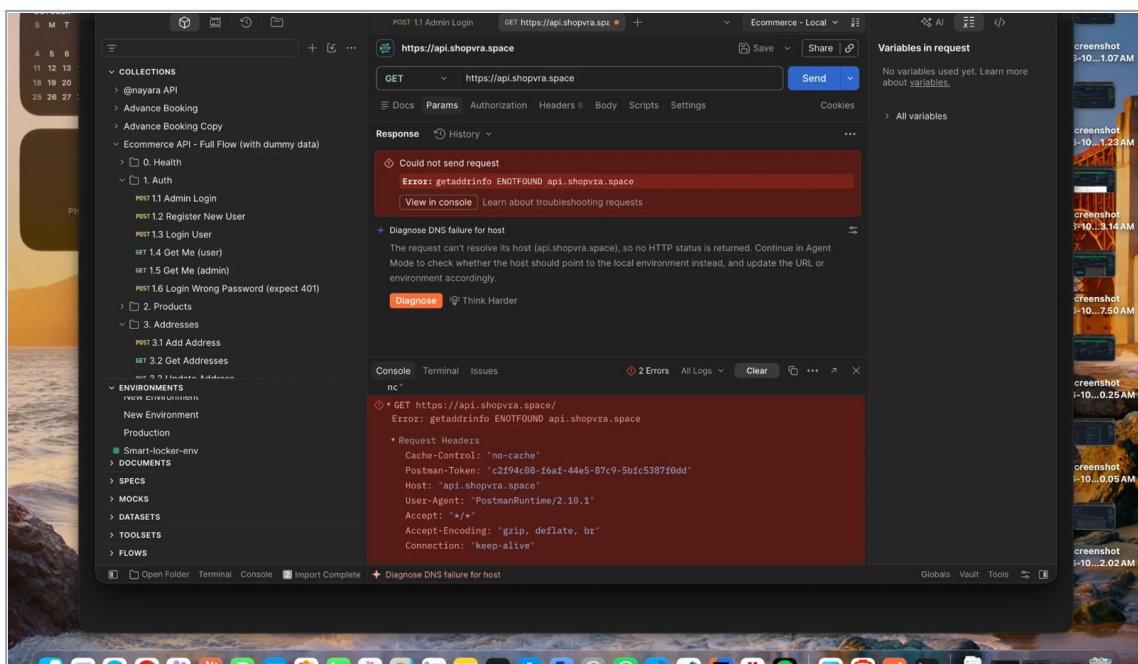


Figure 25. Postman shows the same cached DNS error: getaddrinfo ENOTFOUND api.shopvra.space.

The record is correct (Google 8.8.8.8 and Cloudflare 1.1.1.1 already return it). Only your network's DNS cache is stale. Fix:

- 1 Use public DNS: System Settings > Wi-Fi > Details > DNS > add **1.1.1.1** and **8.8.8.8** > OK > Apply.
- 2 Clear the Mac's DNS cache (asks for your Mac password):

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```
- 3 Chrome: open chrome://net-internals/#dns > **Clear host cache**; chrome://net-internals/#sockets > **Flush socket pools**. Restart Postman.
- 4 Or simply wait: Route 53 limits the cached 'not found' answer to 15 minutes (some providers keep it a little longer).
- 5 Meanwhile the CloudFront domain works: <https://d2czc93szwjsn9.cloudfront.net>

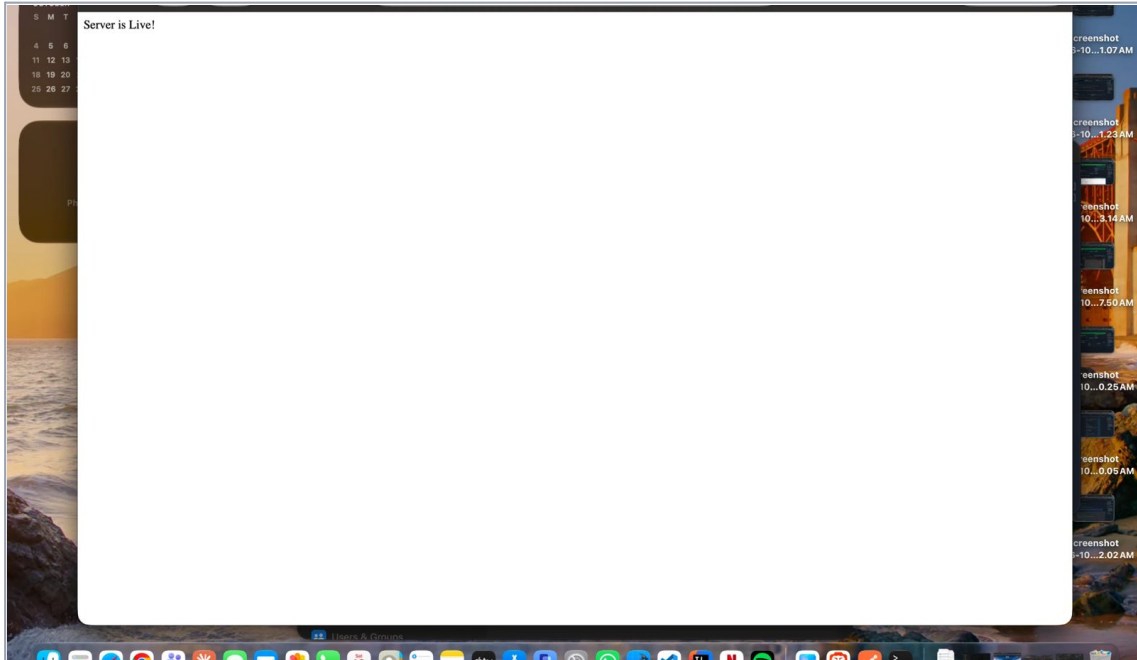


Figure 26. After the DNS cache cleared: <https://api.shopvra.space> shows 'Server is Live!'.

Visitors and recruiters are not affected - their devices never looked up api.shopvra.space before the record existed.

11. Point the web app and mobile app at the API

Both apps are already updated in code; the production URL is committed in code/config, so it works even on a fresh clone.

App	How it chooses the backend
Web app	<code>src/context/ShopContext.jsx</code> : production build -> https://api.shopvra.space ; <code>npm run dev</code> -> http://localhost:3000 (from <code>.env.development</code>). Override with <code>VITE_BACKEND_URL</code> .
Mobile app	<code>constants/api.ts</code> : release builds -> https://api.shopvra.space ; <code>npx expo start</code> -> backend on the PC running Expo. <code>eas.json</code> sets <code>EXPO_PUBLIC_API_URL</code> for preview and production builds.

Deploy the web app

```
cd frontend/webapp_ecommerce_app
npm install
npm run build          # dist/ calls https://api.shopvra.space

aws s3 sync dist s3://<your-web-bucket> --delete
aws cloudfront create-invalidation --distribution-id <WEB_DISTRIBUTION_ID> --paths "/*"
```

AWS Console › CloudFront › (web app distribution, not ecommerce-backend) › Error pages › Create custom error response

HTTP error code **403 (then repeat for 404)**

Customize error response **Yes**

Response page path **/index.html**

HTTP response code **200: OK**

Needed so refreshing /orders or /profile, and Stripe's return to /verify, load the React app.

Build the mobile app

```
cd frontend/mobile_ecommerce_app
eas build -p android --profile preview # APK that calls https://api.shopvra.space
```

To test against AWS while developing, uncomment `EXPO_PUBLIC_API_URL=https://api.shopvra.space` in `mobile_ecommerce_app/.env` and run `npm run expo start -c`.

Postman

In the 'Ecommerce - Local' environment set `baseUrl = https://api.shopvra.space/api` and `serverUrl = https://api.shopvra.space`; type the admin password into Current value; Run collection.

12. Stripe webhook

Stripe Dashboard › Developers › Webhooks › Add endpoint (test mode)

Endpoint URL **https://api.shopvra.space/api/stripe**

Events **payment_intent.succeeded, payment_intent.payment_failed, payment_intent.canceled**

After saving **Reveal the Signing secret (whsec_...) and set it as STRIPE_WEBHOOK_SECRET in Lambda**

Test card: 4242 4242 4242 4242, any future date, any CVC. After payment the webhook marks the order paid and clears the cart.

13. Budget alert and logs

AWS Console › Billing and Cost Management › Budgets › Create budget

Budget setup **Use a template (simplified)**

Template **Monthly cost budget (or Zero spend budget)**

Budgeted amount **1.20 USD (about Rs 100)**

Email recipients **your email - alerts at 85% and 100%**

The first two budgets are free.

AWS Console › CloudWatch › Log groups › /aws/lambda/ecommerce-api › Actions › Edit retention setting

Retention **1 week (default is Never expire)**

Lambda › ecommerce-api › Monitor › View CloudWatch logs. Every request is logged as [req] METHOD /path -> status.

Keep the API fast for recruiters (optional): a free uptime monitor (e.g. UptimeRobot) calling `https://api.shopvra.space/` every 5 minutes avoids the 1-3 second cold start, and stays inside the free tier.

14. Deploying updates later

- 1 Change code and test locally: `npm start`.
- 2 Build: `npm run build:lambda`.
- 3 Lambda > ecommerce-api > Code > Upload from > .zip file > lambda.zip > Save. Live within seconds - no CloudFront change needed.
- 4 Or with the AWS CLI:

```
aws lambda update-function-code --function-name ecommerce-api --zip-file fileb://lambda.zip
```

- 5 Environment variable changes apply to new requests automatically.

15. Cost

Item	Free allowance	Portfolio usage	Monthly
Lambda	1M requests + 400,000 GB-s (always free)	a few thousand requests	Rs 0
CloudFront (Free plan)	\$0/month plan	low traffic	Rs 0
WAF (monitor mode)	included in Free plan	-	Rs 0
CloudWatch Logs	5 GB	a few MB	Rs 0
ACM certificate	free	1 wildcard	Rs 0
Route 53 hosted zone	-	1 zone (shared with web app)	~Rs 44
S3 (web app files)	-	a few MB	~Rs 1-5
MongoDB Atlas M0, Cloudinary free	free plans	demo data	Rs 0
Total			~Rs 45-60

Approximate (about Rs 88 per USD). GoDaddy domain renewal is billed separately each year.

Speed tip: live requests take about 1-2 s. Check your Atlas cluster region (Atlas > Database > cluster). If it is not N. Virginia (us-east-1), every database query crosses continents. For users in India, Atlas and Lambda both in Mumbai (ap-south-1) would be fastest; CloudFront and the certificate stay as they are.

16. Troubleshooting

Symptom	Cause and fix
Browser: DNS_PROBE_FINISHED_NXDOMAIN ; Postman: ENOTFOUND	DNS record missing or stale cache. Set up routing (step 9); clear DNS cache / use 1.1.1.1 (step 10).
Forbidden / 403 on api.shopvra.space	Origin request policy must be AllViewerExceptHostHeader (step 8).
Login/cart/orders fail with 403 after CloudFront	'Allow private Lambda function access' was enabled - disable it, keep Function URL auth NONE.
Image upload returns 403	WAF blocking large bodies - turn on monitor mode in the distribution's Security tab.
502 / 504	Check CloudWatch logs: usually Atlas Network Access (0.0.0.0/0), wrong MONGODB_URI, or timeout too low (30 s).
Browser CORS error	CORS_ORIGINS must contain the exact origin (e.g. https://shopvra.space, no trailing slash). Keep Function URL CORS unchecked.
'Email and password are required' on Lambda	Old zip without the Express 5 body fix - rebuild with <code>npm run build:lambda</code> and upload.
413 / upload too large	Lambda request limit about 6 MB - keep images under ~4 MB per upload.

Symptom	Cause and fix
Certificate not listed	Must be in us-east-1 and cover the name (*.shopvra.space covers api.shopvra.space).
First request slow	Cold start 1-3 s - normal; use an uptime monitor (step 13).

17. Final checklist

Step	Status
Code prepared, lambda.zip built and tested (15/15)	Done
Lambda ecommerce-api created, zip uploaded, 512 MB / 30 s	Done
Environment variables set; CORS_ORIGINS and NODE_ENV	Done
JWT_SECRET replaced with a real random value	To do
Leaked secrets rotated (MongoDB password, Cloudinary secret, Stripe key) and updated in Lambda	To do
Function URL (NONE) - 'Server is Live!'	Done
CloudFront ecommerce-backend (Free plan, CachingDisabled, AllViewerExceptHostHeader, WAF monitor mode)	Done
Route 53 api records (A + AAAA alias)	Done
https://api.shopvra.space live - 18/18 shopper flow	Done
Web app built with the new API, uploaded to S3, cache invalidated, 403/404 -> /index.html	Built - upload to do
Mobile APK built (eas build -p android --profile preview)	To do
Stripe webhook -> /api/stripe and STRIPE_WEBHOOK_SECRET in Lambda	To do
AWS Budget alert (about Rs 100) and log retention 1 week	To do